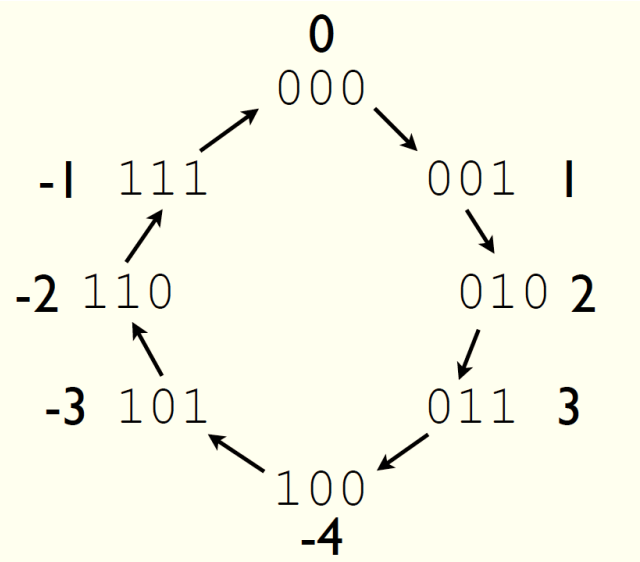



```
  0xCA = 1100 1010
+ 0xE7 = 1110 0111
       ---------
       1011 0001
C = 1
V = 0
```

Lecture 02:
Binary Arithmetic

CMPSC 64, SPRING 2026

ZIAD MATNI, PH.D.

UNIVERSITY OF CALIFORNIA, SANTA BARBARA

Administrative

How many of you do ***NOT*** have access to this course's:

- Canvas?
- Gradescope?
- Piazza?
- CoE (Unix) Account?
- iClicker?

CANVAS FRONT PAGE NOW HAS ALL **OFFICE HOURS** LISTED!

DSP Students

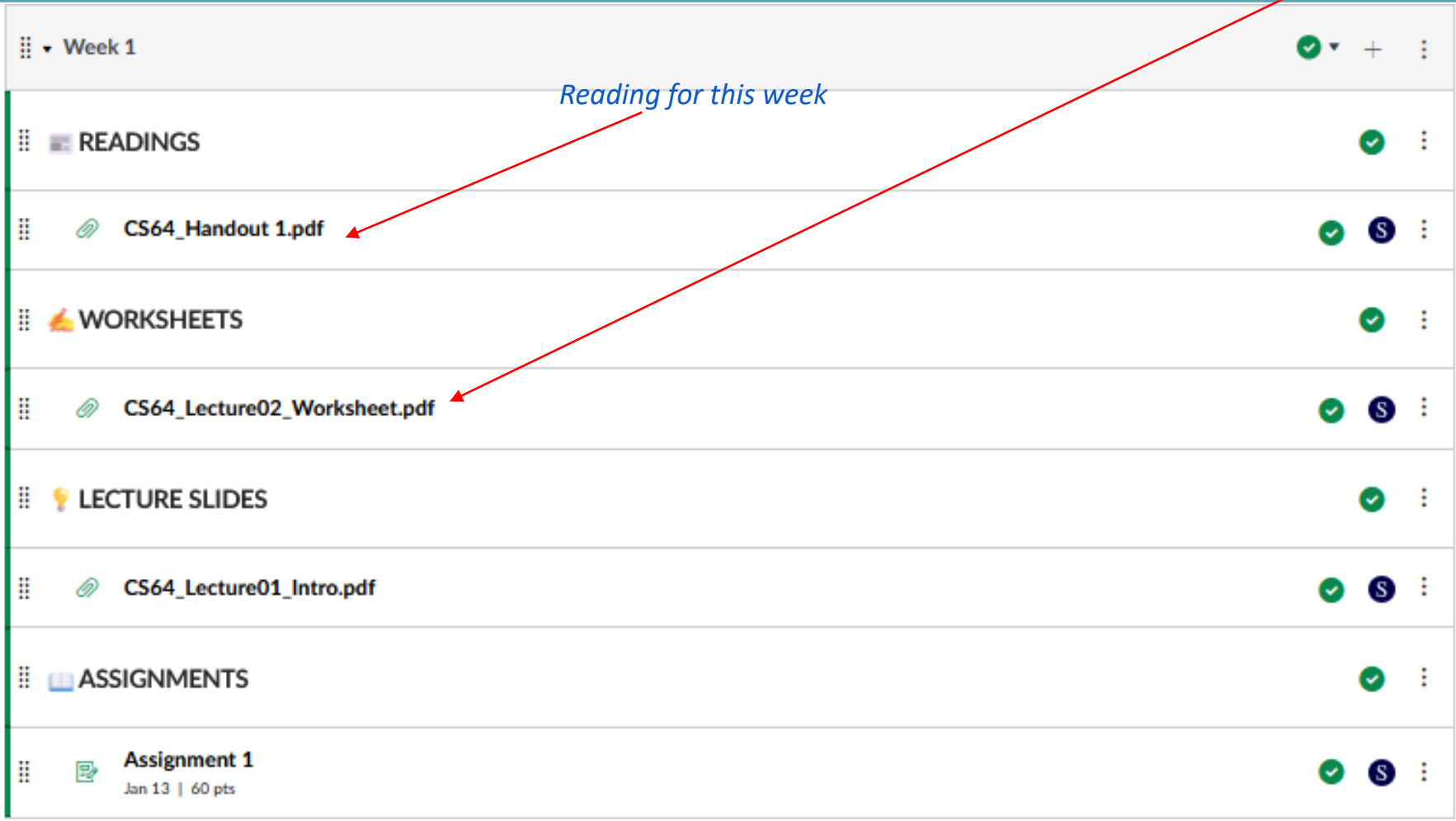
Remember: Assignment #1 due on ***Monday by 11:59 PM*** on **Gradescope**

Attendance sheet

Canvas → Modules

Worksheet

When you see this, you can take notes in the provided worksheet for the lecture!

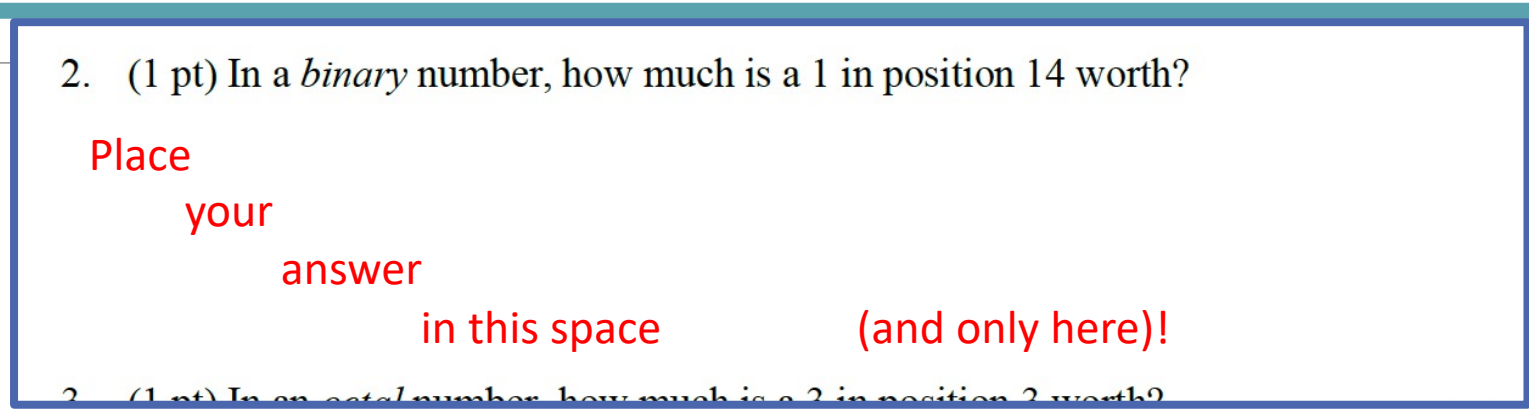


RE: The Proper Use of Gradescope!

Place your answers

only in the spaces provided

in the homework template!



Preference: Use a **PDF editing tool** to insert your answer

- See Piazza post about this
- Neatness counts!

Worksheet

It's Convenient to Remember the Powers of 2

Power of 2	Value
2^1	**2**
2^2	**4**
2^3	**8**
2^4	**16**
2^5	**32**
2^6	**64**
2^7	**128**
2^8	**256**
2^9	**512**
2^{10}	**1024**

Worksheet

iClicker!

Note: **0x** means this is a hexadecimal number!!

Use positional notation to find the decimal equivalent of **0x123**

A. 81624

B. 443

C. 291

D. 123

E. 99

123 in base 16:

2 1 0 ← *position of digits*

$$\mathbf{1} \times 16^2 + \mathbf{2} \times 16^1 + \mathbf{3} \times 16^0$$

$$= 256 + 32 + 3$$

$$= \underline{\mathbf{291}}$$

Converting **Binary** *to* Octal and Hexadecimal

(or any base that's a power of 2)

NOTE THE FOLLOWING:

Binary is	1 bit per digit (because there's **2** digits: 0 or 1)	$2^1 = 2$ *(duh!)*
Octal is	3 bits per digit (b/c there's **8** digits: 0 thru 7)	$2^3 = 8$
Hexadecimal is	4 bits per digit (b/c **16** digits: 0 thru F)	$2^4 = 16$

RULES: Use the "***group the bits***" technique to convert **bin** or **oct** to **hex**

- Always start from the ***least significant digit***
- Group every **3 bits** together for **bin → oct**
- Group every **4 bits** together for **bin → hex**

Worksheet

Converting **Binary** *to* Octal and Hexadecimal

Take the example: ***10100110*** and convert it into

… octal (group in 3s):

1 0	1 0 0	1 1 0
2	***4***	***6***

246 in octal

REMEMBER:
Start your grouping from the Least Significant Bit (LSB)!!!

… hexadecimal (group in 4s):

1 0 1 0	0 1 1 0
10	***6***

0xA6 in hexadecimal

Worksheet

Converting **Octal and Hexadecimal** *to* Binary

Even easier!!… Each *separate digit* is converted into bits!!

Examples:

1. **741** in Octal - remember: every *octal* digit is *3 bits*!
   - Ans: `111100001`

2. **0x741** in Hex - remember: every *hex* digit is *4 bits*!
   - Ans: `011101000001`

Worksheet

iClicker!!

What's the **hexadecimal** representation of the binary **101101101**?

A. 0xB61

B. 0x16D

C. 0x114

D. 0x555

101101101 in hex is the same as:

0001 0110 1101

= **0x16D**

Converting **Decimal** *to* Other Bases

Algorithm for converting number in base 10 to other bases

While (the **quotient** is **not zero**): // i.e. repeat until your quotient is **zero**

1. Divide the decimal number by the **new base**
2. Make the remainder the next digit to the **left** in the answer
3. Replace the original decimal number with the **quotient**

***Example*: What is *88 (base 10)* in base *8*?**

ANSWER: 130

88 / 8 = 11 R 0
11 / 8 = 1 R 3
1 / 8 = 0 R 1

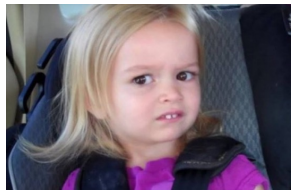
Sanity Check – does the reverse work?

$(130)_8 = 1*8^2 + 3*8^1 + 0*8^0$

$= 64 + 24 = \mathbf{88}$

Why do CPU programmers celebrate Christmas and Halloween on the same day?

Because Oct-31 = Dec-25 !!!



Negative Numbers in Binary

The **Two's Complement Method**

Let's write out $\mathbf{-6_{(10)}}$ in 2s-Complement binary in **4 bits**:

First take the unsigned (abs) value (i.e. +6) and convert to binary: **0110**

Then reverse all the bits (i.e. make them opposites): **1001**

Finally, add 1: **1010**

So, $-6_{(10)} = 1010_{(2)}$ according to this rule

Let's do it Backwards… By doing it THE SAME EXACT WAY!

*2's-Complement is a **reversible algorithm**!*

Take **1010** from our previous example (which was -6)

Reverse the bits & it becomes **0101**

Now add 1 to it & it becomes **0110**, which is $+6_{(10)}$

Trailing 0s and 1s

Note:

In positive binary numbers, trailing 0s to the left do not change the number

EXAMPLE: `011 = 00000000011` (still the same as **3**!!!)

Likewise:

In negative binary numbers, trailing 1s to the left do not change the number

EXAMPLE: `101 = 11111111101` (still the same as **–3**!!!)

Worksheet

iClicker!!!

REMEMBER:
2's Complement method:
- **Reverse all bits**
- **Add 1**

What's the **2's complement** expression for **–4**, as written in 4-bits?

A. 1011
B. 1100
C. 1101
D. 0101
E. 0100

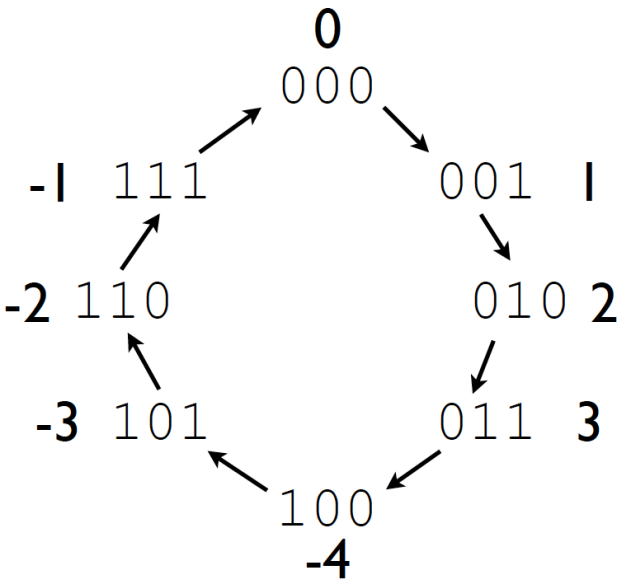
In 4-bit, **4** is written as: **0100**
The 2's complement (i.e. **–4**) is:

a) Reverse the bits: 0100 → 1011
b) Add 1: 1011 + 1 → **1100**

Worksheet

Range of Integers

*Example of signed numbers in **3-bits** →*



The *range* of integers represented by ***N*** bits *differs* between unsigned and signed binary numbers.

Given **N** bits, the range represented is:

0 to $+2^N - 1$ *for positive numbers (aka **unsigned integers**)*

and -2^{N-1} to $+2^{N-1} - 1$ *for numbers that can be negative/positive (aka **signed integers** – this is where we use 2's complement)*

Worksheet

iClicker!!!!

REMEMBER:
Negative no. range is:
-2^{N-1} to $2^{N-1} - 1$
N is number of bits

How many bits, at minimum, do we need to express **–31** ?

A. 3 bits
B. 4 bits
C. 5 bits
D. 6 bits
E. 7 bits

Note that:
$-31 = -32 + 1$, i.e., $\mathbf{-31 = -2^5 + 1}$

We know that for negative binary numbers, the range is:
-2^{N-1} to $2^{N-1} - 1$

So, we need the following to be true:
$-2^{N-1} \leq -2^5 + 1 \leq 2^{N-1} - 1$

So, AT MINIMUM: $N - 1 = 5$, i.e., $\mathbf{N = 6}$

Worksheet

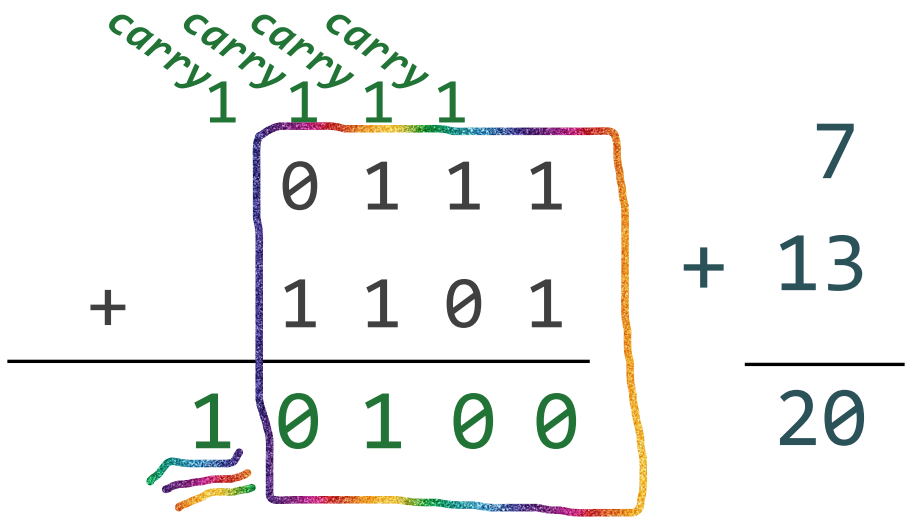
Addition in Binary

```
          carry carry
            1   1
          0 0 1 1        3
    +     1 0 1 1    + 11
  ---------------    ------
          1 1 1 0       14
```

Note: All carries were internal to (contained in) the 4-bits, i.e., no "carrying-out" outside the 4-bit "boundary"

Worksheet

Addition in Binary



*A CPU must use a **fixed** number of binary digits (bits) for certain values.*

*e.g., regular **integers** in CPUs are represented with **32-bits** – no more, no less!*

WHY do you think that is??!!

What implications does this have?

Worksheet

Exercise 1

*Implementing an **8-bit adder**:*

What is 0x52 + 0x8B ?

Assume ***unsigned*** numbers.

Give answer in hex
and identify the *carry out* bit

```
  0x52 = 0101 0010
+ 0x8B = 1000 1011
-------------------
       0 1101 1101
```

ANSWER:

0xDD

With *no* carry out!

So, the carry out bit, C = 0

Worksheet

Exercise 2 -- Students

*Implementing an **8-bit adder**:*

What is 0xCA + 0x67 ?

Assume ***unsigned*** numbers.

Give answer in hex
and identify the *carry out* bit

A little sanity-check…
*8-bit **unsigned** number range:*
*0 to 2^8-1, i.e., 0 to **+255***

0xCA = 202
0x67 = 103
*So, their sum = **305***
Conclusion?????

```
  0xCA = 1100 1010
+ 0x67 = 0110 0111
-------------------
       1 0011 0001
```

ANSWER:

0x31

***with* a carry out bit, C = 1**

Carry Out vs. Overflow in Binary Addition

Going *Out of Range* in CPU Binary Addition

For ***unsigned*** numbers, when the **Carry Out bit**, **C** (sometimes called C_{OUT}) = **1**,
it means that the result **did not fit into the number of bits allotted**
in other words, the result is out of range

But what if I am using ***signed*** numbers, i.e. numbers in 2's complement?

- ANSWER: You'll need a different indicator than **Carry Out** (bit **C**) !!!

Rule: To see if we've gone out of range with our addition:

1. When adding ***unsigned numbers*** then **carry out (C)** bit is what we look for
2. When adding ***signed numbers*** then **overflow (V)** bit is what we look for

Overflow Example

Suppose I'm adding two ***signed*** numbers: 1001 + 1011

- That's (-7) + (-5) ➔ I expect the answer to be **-12**

```
   1001
+  1011
------
= 1 0100
```

← **The 4-bit answer is +4 !!! That's not correct!!!!!!!!!!!!!**

Overflow Example

When I added **signed numbers** 1001 + 1011, I expected -12, but I got +4

In fact, **I added 2 negative numbers and got a positive sum** – ***Whaaaaat??***

This error illustrates that I've gone beyond the scope/range of 4-bit ***signed numbers***

In other words, the expected answer (-12) is beyond the range given by 4 bits in 2's complement!!!

- *Note*: 4-bit 2's complement (i.e. signed number) range is **-8 to + 7** *(yup, checks out)*

Worksheet

How Do We Determine if **Overflow** Has Occurred?

RULE: When adding 2 ***signed*** numbers: ***x + y*** to get a sum ***s***

if **x, y > 0** AND **s < 0**

OR if **x, y < 0** AND **s > 0**

Then, **OVERFLOW!** has occurred

Worksheet

Exercise 1

Add -39 and 92 in *signed 8-bit binary*
& identify the C and V bits

Sanity-check

```
        -39    1101 1001
         92    0101 1100
        ---    ---------
         53   10011 0101
```

That's 53 in signed 8-bits! Looks ok!

There is a carry-out bit (**but we don't care** – ***why?***)

AHA! But there is **no** overflow (V) (***how do we know this?***)

Note that the 2 numbers are signed and **one is neg.** and **one is pos.** ➔ you cannot get overflow this way!

Side-note:
What is the range of signed numbers w/ 8 bits?

-2^7 to $(2^7 - 1)$, or
-128 to +127

Worksheet

Exercise 2 -- Students

Add: 104 and 45 in 8-bit binary & identify the C and V bits

(note, I did not specify if they're signed or unsigned!)

```
104          0110 1000
 45        + 0010 1101
---        ----------
149        0 1001 0101
```

*NOTE: That's a negative number, i.e. it's **NOT** 149*

There's no carry-out (so **C = 0**)

But there **is** overflow!

We added 2 positive numbers and got a negative number! So, **V = 1**

YOUR TO-DOs

Check Piazza every day! ☺

Do your reading (Handout #1)

Go to your Lab Section on Friday

Work on Assignment #1

- Submit it as a **PDF** using ***Gradescope***
- Due on **Monday by 11:59 PM**

</LECTURE>